Programming Assignment: Implementing Dijkstra's Algorithm with Apache Spark on Azure VMs

Github: https://github.com/laosrb/Cloud-Computing-VM-Project

1. Implementation

Dijkstra's algorithm is sequential: it repeatedly pops the single closest unvisited vertex from a priority queue and relaxes outgoing edges. That pattern has no natural parallel decomposition, so it does not even map cleanly onto Spark's data-parallel RDD model, where every executor works on an independent partition of the data with no shared mutable state as a global priority queue.

Instead, this implementation performs Dijkstra's core relaxation rule in parallel, across every edge, in synchronous rounds:

dist[v] = min(dist[v], dist[u] + weight(u, v))

Every round is one Spark stage made from three RDD transformations: a join between the current distances RDD (node, dist) and the edges RDD (u, (v, w)) on the source vertex; a map that creates a candidate distance for each edge's target vertex and a reduceByKey(min) that merges those candidates with the existing distances, keeping the smallest value seen for each node. This repeats until no distance changes (convergence) or a safety bound of num_nodes - 1 rounds is reached the same bound Bellman-Ford relies on to guarantee termination. On graphs with non-negative edge weights, this relaxation process converges to exactly the same shortest-path distances that classic Dijkstra would produce, because both approaches are grounded in the same underlying relaxation invariant; they only differ in the order and scale in which relaxations are applied.

The job (dijkstra_spark.py) takes an input file path, a source node id, and an optional output path. It parses the edge-list format described in the assignment, builds the edges RDD, runs the iterative relaxation loop described above, and prints/writes distances in the format, replacing INF for unreachable nodes.

2. Performance Analysis

Correctness and convergence speed were verified against the assignment's required 10,000-node / 100,000-edge test graph) using a local reference implementation of classic heap-based Dijkstra as ground truth for the same relaxation logic the Spark job runs.

Metric	**Value**	**Notes**
Nodes	10,000	weighted_graph.txt
Edges	100,000	directed, weights 1-20
Convergence bound	9,999 rounds	num_nodes - 1, Bellman-Ford-style worst case
Actual rounds to converge	16 rounds	measured; equals graph's shortest-path depth from source
Reachable nodes	10,000 / 10,000	test graph built with a guaranteed spanning structure
Correctness check	0 mismatches	vs. heap-based reference Dijkstra, all 10,000 nodes

The key result is that the algorithm converged in 16 rounds rather than the 9,999-round worst-case bound. For graphs where the number of edges hops on the shortest path from the source to the farthest reachable node (the shortest-path depth) is much smaller than the number of nodes, this relaxation approach converges quickly each round advances the known shortest-path frontier by one hop, so the number of rounds needed is bounded by that depth, not by the node count. This is the same property that makes Bellman-Ford practical on typical graphs despite its pessimistic worst-case bound.

In a real Spark cluster deployment, per-round cost is dominated by the join and reduceByKey shuffle across the edges RDD, so overall wall-clock time scales with (number of rounds) x (shuffle cost per round). For the required 10,000-node / 100,000-edge graph, this keeps total shuffle volume modest even across multiple executors. On larger graphs, caching the edges RDD (as the implementation does) avoids re-reading and re-parsing the input file every round, and using enough partitions to match the executor core count keeps parallelism high without excessive scheduling overhead.

Practical levers for tuning cluster performance: increasing total-executor-cores and executor-memory so more of each round's join/reduceByKey work runs concurrently; setting a higher spark.default.parallelism so the edges and distances RDDs are partitioned finely enough to use all available cores; and, for graphs where the shortest-path depth is large relative to node count, adding an early-exit convergence check (already implemented) to avoid wasted rounds once distances stabilize.

3. Challenges and Lessons Learned

- Dijkstra is greedy, strictly ordered vertex selection does not parallelize directly; the main design decision was recognizing that its relaxation step not its priority-queue traversal order is the part that generalizes to a distributed setting, at the cost of doing more relaxation work than a sequential priority-queue Dijkstra would.
- Convergence detection matters for efficiency: running a fixed num_nodes - 1 rounds every time (the safe worst-case bound) would waste enormous amounts of cluster time on typical graphs; tracking whether any distance changed each round and stopping early cut the required rounds from a worst-case 9,999 down to an actual 16 on the test graph.
- RDD caching of the edges RDD was necessary without it, Spark would re-read and re-parse the input file from scratch on every round's join, since RDD transformations are lazy and recomputed by default.
- Interestingly, the assignment PDF's own 5-node worked example contains an inconsistency: its sample output lists a distance of 10 for node 3, but the only path into node 3 in its sample directed edge list is 0 -> 1 -> 3 (7 + 9 = 16); a distance of 10 is not reachable from that edge list under standard directed-edge semantics. This implementation was cross-checked by hand against that example (see verify_logic.py) and

correctly returns 16 for node 3, matching the PDF's own listed values for nodes 0, 1, 2, and 4.